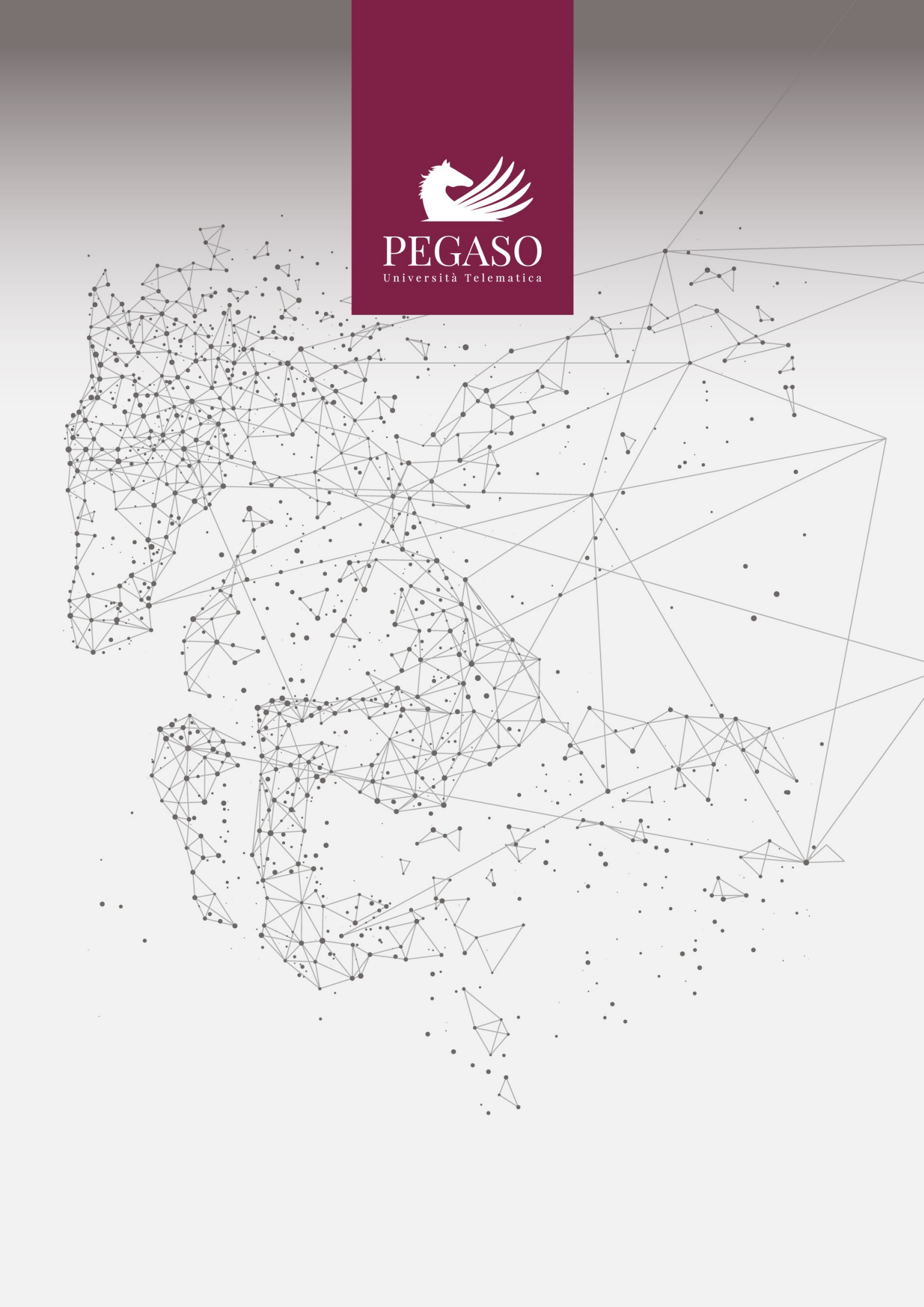

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. LA CHIAMATA DI PROCEDURA 3
2. LE ISTRUZIONI ASSEMBLY MIPS JAL E JR 5
3. ANNIDAMENTO DI CHIAMATE DI PROCEDURA E CALL STACK 7
4. FORMATO E MODALITÀ DI INDIRIZZAMENTO DI JAL 9
5. FORMATO E MODALITÀ DI INDIRIZZAMENTO DI JR 14
BIBLIOGRAFIA 19

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. La chiamata di procedura

Le *procedure* (o *funzioni* se restituiscono valori) costituiscono una metodologia utilizzata dai programmatori per permettere il riuso del codice e per rendere più chiara l'organizzazione dei programmi.

Nel paradigma della programmazione *procedurale* sono utilizzate per organizzare il programma in parti indipendenti che eseguono compiti specifici.

Nei linguaggi ad Alto Livello basati sul paradigma di programmazione *object-oriented*, come Java, il concetto di procedura è assimilabile a quello di *metodo*.

Le procedure sono identificate da un *nome* e da una lista di *parametri* su cui operano.

La presenza in un punto del programma in esecuzione del nome di una procedura determina una **chiamata della procedura**, ovvero, il passaggio alla esecuzione della lista di istruzioni della procedura con tale nome. La comunicazione tra il *programma chiamante* e la *procedura chiamata* avviene attraverso la lista dei *parametri* associata alla definizione della procedura.

Al termine dell'esecuzione della procedura possono essere eventualmente restituiti dei *valori*, e si riprende l'esecuzione del programma chiamante dal punto in cui è avvenuta la chiamata della procedura.

A livello Hardware le istruzioni in Linguaggio Macchina che traducono una procedura sono tipicamente memorizzate in locazioni di Memoria separate dal blocco del programma chiamante, quindi la **chiamata** della procedura è eseguita mediante una istruzione di **salto** alla esecuzione di queste istruzioni.

Per eseguire il salto all'esecuzione di una procedura è necessario porre nel registro *Program Counter* l'indirizzo della prima istruzione della procedura.

Un'ulteriore istruzione di **salto** posta alla fine della procedura consente di **riprendere l'esecuzione** del programma chiamante dal punto in cui è avvenuto il salto alla procedura.

Per eseguire questo salto di ritorno occorre porre nel registro *Program Counter* l'indirizzo dell'istruzione successiva a quella che ha determinato la chiamata della procedura, detto **indirizzo di ritorno**.

Inoltre, occorre stabilire un meccanismo di comunicazione tra i due programmi per effettuare il passaggio dei parametri dal programma chiamante alla procedura e per consentire il reperimento degli eventuali risultati restituiti dalla procedura.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In sintesi, per l'esecuzione di una procedura occorre, in generale, effettuare le seguenti operazioni:

programma chiamante

- memorizza i parametri in locazioni note alla procedura,
- salva l'indirizzo di ritorno, cioè dell'istruzione successiva a quella che ha effettuato la chiamata,
- passa all'esecuzione della procedura,

procedura

- utilizza i parametri passati per eseguire le istruzioni,
- memorizza i risultati in locazioni note al programma chiamante,
- torna all'esecuzione del programma chiamante utilizzando l'indirizzo di ritorno.

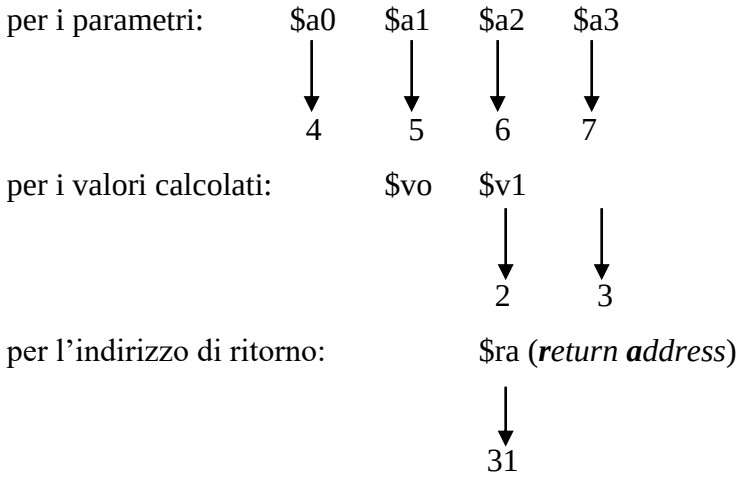
Nel paragrafo successivo vengono descritte le istruzioni Assembly MIPS che consentono di gestire le chiamate di procedura.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Le istruzioni Assembly MIPS jal e jr

Nell'Architettura MIPS, per la memorizzazione dei dati relativi ad una chiamata di procedura (parametri, valori ritornati, indirizzo di ritorno) vengono utilizzati dei registri del processore riservati a questo scopo, i cui indirizzi in notazione simbolica e decimale sono:



Per la gestione delle chiamate di procedura, il Linguaggio Assembly MIPS include una specifica **istruzione di salto** denominata **JUMP AND LINK** (salta e collega), con Codice Operativo *jal*, che nel programma chiamante effettua il salto all'esecuzione della procedura:

jal *nome_procedura*

L'esecuzione dell'istruzione di salto *jal* effettua le seguenti operazioni:

- **salta** all'esecuzione della procedura identificata da *nome_procedura* ponendo nel *Program Counter* l'indirizzo della prima istruzione del relativo programma,
- **salva** contemporaneamente nel registro $ra l'indirizzo di ritorno, ovvero l'indirizzo dell'istruzione successiva a *jal*, che è contenuto nel *Program Counter* al momento dell'esecuzione.

La memorizzazione dell'indirizzo di ritorno nel registro $ra giustifica il termine *link* presente nella denominazione di questa istruzione perché questo indirizzo è il collegamento che consente di riprendere l'esecuzione dal punto in cui è avvenuta la chiamata della procedura nel programma chiamante.

Per riprendere l'esecuzione del programma chiamante l'Assembly MIPS prevede una **ulteriore istruzione di salto** denominata **JUMP REGISTER** (salta tramite registro) con Codice Operativo *jr*, che, posta alla fine della procedura, effettua il salto di ritorno all'esecuzione del programma chiamante:

jr $ra

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'esecuzione dell'istruzione di salto *jr* effettua un salto ponendo nel *Program Counter* l'indirizzo di ritorno contenuto nel registro $ra.

GESTIONE DELLA CHIAMATA IN ASSEMBLY MIPS

I valori dei parametri da passare alla procedura vengono memorizzati nei registri dedicati $a nel **programma chiamante**, prima di effettuare la chiamata di procedura. Quindi si utilizza l'istruzione *jal* che effettua sia il salto alla prima istruzione della procedura, sia il salvataggio dell'indirizzo di ritorno contenuto nel *Program Counter* memorizzandolo nel registro dedicato $ra.

Vengono eseguite le istruzioni della **procedura,** che memorizza gli eventuali valori calcolati nei registri dedicati $v, ed alla fine ritorna alla esecuzione del programma chiamante mediante l'istruzione *jr*.

L'esecuzione di questa istruzione effettua il salto all'indirizzo dell'istruzione che segue *jal*, punto di chiamata della procedura, utilizzando l'indirizzo salvato nel registro $ra.

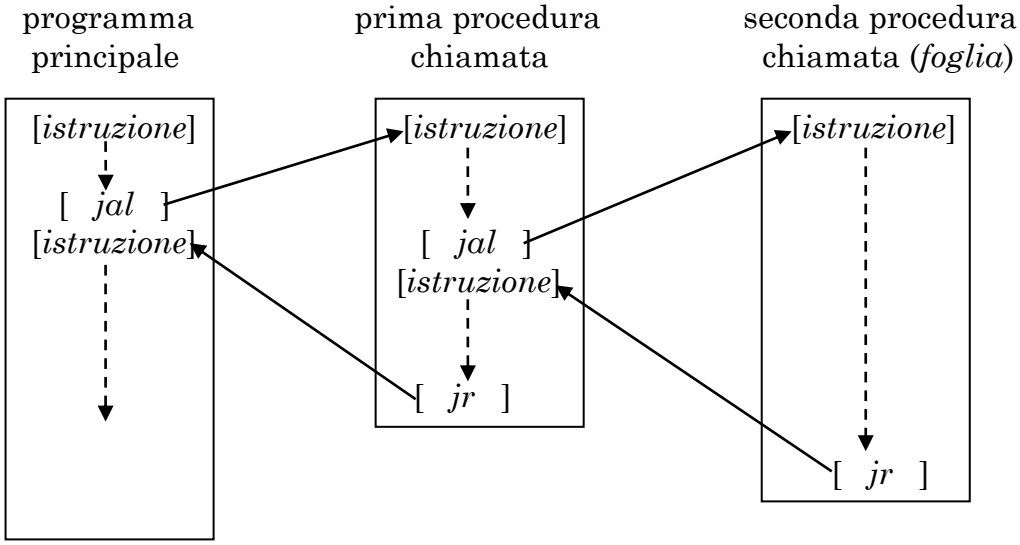
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Annidamento di chiamate di procedura e call stack

Una procedura può a sua volta effettuare una chiamata di procedura nel corso della sua esecuzione, in particolare, in una computazione ricorsiva, una procedura può chiamare se stessa con valori diversi per i parametri.

In questo caso si dice che le chiamate di procedura sono **annidate**. La procedura che non chiama altre procedure è detta *procedura foglia*.



Il meccanismo di comunicazione tra la procedura chiamante e quella chiamata nella traduzione Assembly rimane lo stesso, ma è necessario gestire i diversi insiemi di dati (parametri, risultati restituiti, indirizzi di ritorno) relativi alle varie chiamate di procedura, che vanno conservati mantenendo l'ordine delle chiamate in quanto ogni procedura **chiamante** rimane in attesa della chiusura della procedura **chiamata** per riprendere la sua esecuzione.

Poiché l'ultima procedura che viene chiamata è la prima che restituisce il controllo, la *struttura dati* adatta a memorizzare questi dati è lo *stack*.

Lo ***STACK***, o *pila*, è una **STRUTTURA DATI** costituita da una lista di elementi gestita mediante la modalità LIFO (**L**ast **I**n **F**irst Out), in base alla quale l'ultimo dato inserito è il primo ad essere estratto. In tal modo i dati vengono estratti (letti) in ordine inverso rispetto a quello in cui sono stati inseriti (scritti).

L'ultimo elemento inserito nella lista è detto *top* dello stack.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Gli operatori che si applicano alla struttura dati stack sono:

- *push* (inserimento) aggiunge un dato che diventa il nuovo elemento *top* dello stack
- *pop* (estrazione) preleva l'elemento *top* dello stack e l'elemento che lo precede nella lista diventa il nuovo *top*.

La gestione dello stack è paragonata a quella di una pila di piatti.

Il ***call stack*** (stack delle chiamate) è una struttura dati di Tipo stack associata al programma nella quale sono immagazzinate le informazioni relative alle chiamate di procedura attive in un dato momento, ovvero quelle la cui esecuzione non è terminata.

Le informazioni specifiche di una procedura in esecuzione (parametri, valori ritornati, indirizzo di ritorno) sono memorizzate in un segmento di locazioni successive del *call stack* che viene detto *frame* della procedura.

Generalmente vi è un unico *call-stack* associato ad un programma che occupa una quantità limitata di locazioni di Memoria, fissata di solito all'avvio del programma. La dimensione del *call-stack* dipende dal linguaggio di programmazione, dall'Architettura della macchina, e dalla disponibilità di Memoria.

FLUSSO DI ESECUZIONE

Il programma che effettua la chiamata di procedura memorizza nel *call-stack* i dati relativi alla procedura, tra cui l'indirizzo di ritorno, in un segmento di locazioni detto *frame* a partire dal *top* dello stack.

Quando la procedura chiamata termina la sua esecuzione le informazioni contenute nel relativo *frame* vengono tolte dal *call-stack*, e l'indirizzo di ritorno viene utilizzato per riprendere l'esecuzione dall'istruzione memorizzata a quell'indirizzo.

Se una procedura chiamata chiama a sua volta un'altra procedura, questa inserirà le sue informazioni, tra cui l'indirizzo di ritorno alla procedura chiamante, in un *frame* successivo che determinerà un nuovo *top* del *call-stack*.

Quindi, nel corso del procedere dell'esecuzione, le informazioni delle chiamate di procedura vengono inserite seguendo l'ordine di chiamata nel *call-stack*, e vengono rimosse nell'ordine inverso.

Se viene usato tutto lo spazio destinato al *call-stack* e non si possono più inserire le informazioni di ulteriori chiamate di procedura, si verifica un errore detto *call-stack overflow* che determina l'interruzione dell'esecuzione del programma.

Nei Linguaggi di programmazione di Alto Livello, la gestione del *call-stack* è nascosta al programmatore, che non ha accesso diretto alle locazioni di Memoria occupate dal *call-stack*, né alla gestione delle operazioni di *push* e *pop* su tale struttura dati.

I Linguaggi Assembly, invece, richiedono ai programmatori la gestione del *call-stack*. I dettagli di tale gestione dipendono dalle istruzioni Assembly disponibili e dal Sistema Operativo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Formato e Modalità di Indirizzamento di jal

L'istruzione che traduce in **Linguaggio Macchina** MIPS l'istruzione Assembly ***jamp and link*** utilizza:

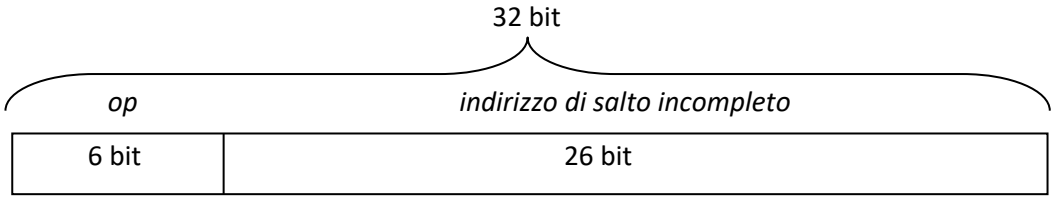
- Formato di Tipo J
- Indirizzamento pseudodiretto

quindi la traduzione e la determinazione dell'indirizzo di salto sono del tutto analoghe all'istruzione di salto incondizionato *Jump*, da cui il Formato J prende la denominazione, e che vengono qui richiamate.

L'unica informazione da fornire per l'esecuzione dell'istruzione *jal* è l'indirizzo di Memoria da cui partono i 4 Byte che rappresentano l'istruzione a cui saltare, che è la prima della procedura chiamata.

Il seguente schema indica la **partizione** concettuale (non fisica) in sottosequenze della sequenza di 32 bit dell'istruzione:

Formato di Tipo J



op: 6 bit, campo del *Codice Operativo* presente in tutte le istruzioni. Indica l'operazione da eseguire ed il Formato dell'istruzione,

indirizzo di salto incompleto: 26 bit, parte della sequenza di 32 bit che costituisce l'indirizzo di salto.

Il Codice Operativo è codificato nel campo di 6 bit comune a tutti i Formati. Per l'istruzione ***jal*** il campo ***Codice Operativo*** contiene il **valore decimale 3**, rappresentato in binario dalla sequenza 000011, e fornisce tutte le informazioni che specificano l'operazione da eseguire ed il Formato dell'istruzione.

Nel campo *indirizzo di salto incompleto* la sequenza binaria di 26 bit rappresenta una parte dei 32 bit dell'indirizzo di salto, cioè, l'indirizzo di Memoria del primo dei 4 Byte in cui è memorizzata la prossima istruzione da eseguire.

La determinazione dell'indirizzo di salto completo viene fatta automaticamente dall'Hardware del computer nella fase di esecuzione dell'istruzione in base alla **Modalità di Indirizzamento pseudodiretto** associata all'istruzione *jal*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi l'esempio di istruzione *jal* dell'Assembly MIPS:

jal *nome_procedura*

viene tradotto in Linguaggio Macchina MIPS nella seguente istruzione con Formato di Tipo J:

op	*indirizzo di salto incompleto*
3	*26 bit che fanno parte dell'indirizzo di salto*

La seguente scelta come esempio per i 26 bit dell'indirizzo incompleto nella rappresentazione binaria:

000011	11111111111000000000111111

fornisce la traduzione in Linguaggio Macchina data dalla sequenza:

00001111111111111000000000111111

La sequenza binaria contenuta nei 26 bit dell'istruzione fornisce solo una parte dei 32 bit di un indirizzo di Memoria.

L'indirizzo completo viene costruito aggiungendo ulteriori bit usando la *Modalità di Indirizzamento pseudodiretto* così definita:

INDIRIZZAMENTO pseudodiretto

L'*Indirizzo di Memoria* di 32 bit dell'istruzione a cui saltare si ottiene

concatenando da sinistra a destra le seguenti sottosequenze:

- i quattro bit più significativi del *Program Counter,*
- i 26 bit del *campo indirizzo,*
- *due bit* 00 nelle posizioni meno significative.

L'idea di base di questa Modalità di Indirizzamento è di fornire "direttamente" l'indirizzo di salto nell'istruzione. Tuttavia questo non è possibile in quanto il campo *Codice Operativo* occupa 6 dei 32 bit dell'istruzione, lasciandone disponibili solo 26 bit.

Si ricorre quindi a dei "risparmi" nella indicazione dei 32 bit dell'indirizzo di salto sfruttando delle informazioni sottintese.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La prima operazione di completamento fornisce due dei sei bit mancanti aggiungendo due bit uguali a 0 a destra della sequenza dei 26 bit, occupando le posizioni meno significative dell'indirizzo di salto.

Questa operazione si basa su due osservazioni: da un lato si ha che la rappresentazione dei numeri multipli di 4 in binario è caratterizzata dalla presenza di due bit uguali a 0 nelle posizioni meno significative.

D'altra parte, memorizzando le istruzioni di un programma in locazioni successive a partire da un indirizzo multiplo di 4, si avrà che tutti gli indirizzi delle istruzioni saranno multipli di 4 (poiché una istruzione in Memoria occupa 4 Byte), e presenteranno una coppia di zeri nelle due cifre meno significative.

Memorizzando in questo modo le istruzioni, si può, quindi, evitare di indicare i due bit meno significativi dell'indirizzo dell'istruzione a cui saltare, perché sempre uguali a zero, aggiungendoli automaticamente ai 26 bit contenuti nel campo indirizzo del Formato J.

I rimanenti 4 bit mancanti alla sequenza di 32 dell'indirizzo occupano le posizioni più significative. Nella numerazione, questi bit cambiano quando i bit nelle 28 posizioni meno significative hanno descritto tutte le 2^{28} combinazioni possibili, che individuano un notevole segmento di Memoria costituito da:

$$2^{28} \text{ Byte} = 2^8 \times 2^{20} \text{ Byte} = 256 \text{ Mega Byte.}$$

Poiché ogni istruzione occupa 4 Byte, si ha che un segmento di locazioni di Memoria di questa dimensione equivale a:

$$256 \text{ Mega Byte} = 64 \text{ Mega di } \textit{parole}$$

in cui è possibile memorizzare circa 64 Milioni di istruzioni.

Se tutto il programma viene memorizzato in locazioni di questo segmento di Memoria, tutti gli indirizzi delle istruzioni avranno i 4 bit più significativi che non cambiano, quindi si può evitare di indicarli nel campo indirizzo del Formato J.

Pertanto, la seconda operazione di completamento aggiunge questi bit all'indirizzo di salto copiandoli dal registro *Program Counter* che contiene sempre l'indirizzo della prossima istruzione da eseguire, cioè la successiva a quella in esecuzione.

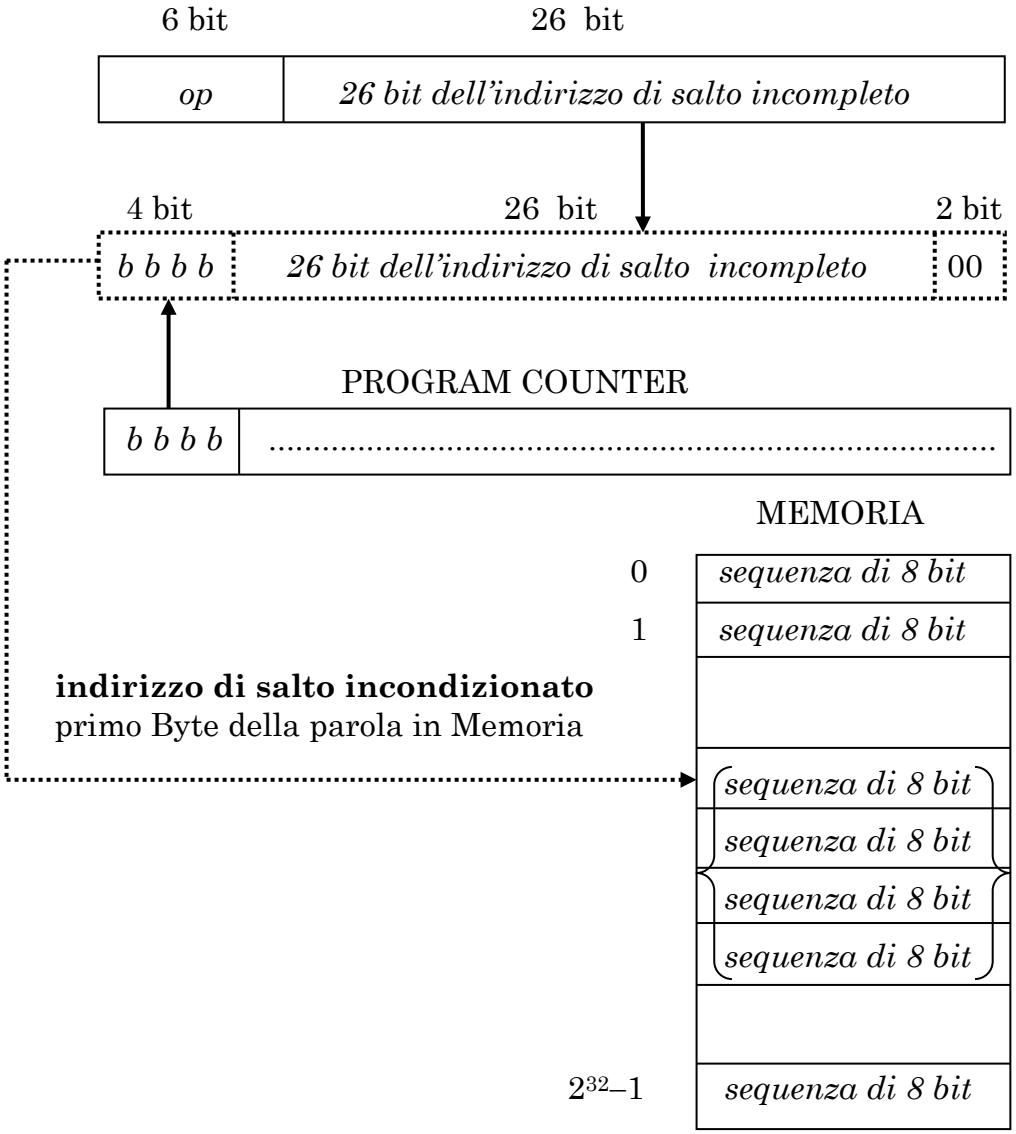
Il completamento dell'indirizzo con queste due operazioni è fatto automaticamente dall'Hardware del computer nella fase di esecuzione dell'istruzione motivando, così, la denominazione di *pseudodiretto* data all'Indirizzamento.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura fornisce una rappresentazione schematica di tale Modalità di Indirizzamento:

MODALITÀ di Indirizzamento *pseudodiretto*



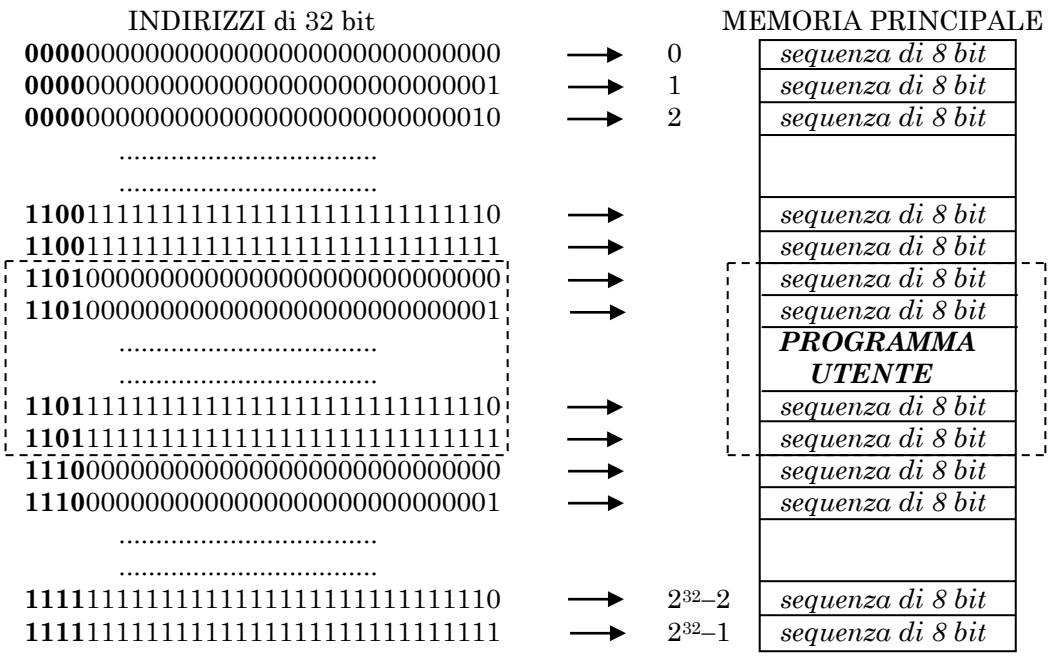
OSSERVAZIONE

L'applicazione della regola di calcolo dell'indirizzo di Memoria con la Modalità di **Indirizzamento pseudodiretto** richiede che tutto il programma sia caricato in un segmento di locazioni di Memoria, anche detto *pagina di Memoria* (area tratteggiata in figura) i cui indirizzi hanno i 4 bit più significativi che non cambiano. Questo consente di copiare questi 4 bit dal *Program Counter* (che contiene l'indirizzo di un'istruzione del programma) per aggiungerli al campo di 26 bit del Formato J dell'istruzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PAGINA DI MEMORIA



Se questo non accade, cioè, se il programma viene memorizzato in segmenti di Memoria in cui i 4 bit più significativi degli indirizzi delle istruzioni cambiano, allora la traduzione dell'istruzione Assembly di salto viene fatta utilizzando più istruzioni in Linguaggio Macchina. Questo caso, detto anche di *salto pagina di Memoria*, determina, quindi, un maggior tempo di esecuzione.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Formato e Modalità di Indirizzamento di jr

L'istruzione che traduce in **Linguaggio Macchina** MIPS l'istruzione Assembly ***jump register*** utilizza:

- Formato di Tipo R,
- Indirizzamento tramite registro,

quindi la traduzione e la regola di Indirizzamento sono del tutto analoghe alle istruzioni Aritmetico-Logiche, e vengono qui richiamate.

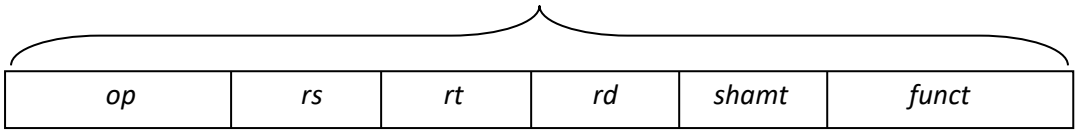
Il Formato di Tipo R deve la sua denominazione al fatto che gli operandi sono contenuti in registri del processore e gli indirizzi che li individuano sono forniti nei campi dell'istruzione. La lettera R rappresenta, quindi, l'iniziale della parola *Registro*.

Gli indirizzi sono numeri interi positivi rappresentati da 5 bit in Notazione posizionale pesata.

Il Formato di Tipo R suddivide i 32 bit della sequenza che rappresenta l'istruzione in 6 sottosequenze in base alla seguente partizione (concettuale, non fisica) in campi di lunghezze e posizioni fissate. Le sottosequenze contenute nei campi sono denotate da nomi simbolici impiegati per descrivere il loro utilizzo nelle istruzioni che usano questo Formato:

FORMATO di Tipo R

32 bit



6 bit 5 bit 5 bit 5 bit 5 bit 6 bit

op: 6 bit, campo *Codice Operativo* presente in tutte le istruzioni, come nel caso delle istruzioni Aritmetico-Logiche specifica solo il Formato dell'istruzione;

rs, rt, rd: 5 bit, contengono indirizzi di registri;

shamt: 5 bit, nelle istruzioni di *shift* contiene il numero di posizioni di scorrimento della sequenza (*shift amount*);

funct: 6 bit, specifica l'operazione (*function*) relativa all'istruzione.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il ***Codice Operativo*** è codificato nel campo di 6 bit comune a tutti i Formati. Per l'istruzione ***jr*** il campo *Codice Operativo* contiene il **valore decimale 0**, rappresentato in binario dalla sequenza 000000, che indica il Formato. La specifica operazione è rappresentata dai 6 bit del campo ***funct*** (funzione) che contiene il **valore decimale 8**, rappresentato in binario dalla sequenza 001000.

Nel caso dell'istruzione *jr* è necessario indicare solo l'indirizzo del registro riservato $ra che contiene l'indirizzo di ritorno all'esecuzione del programma chiamante. A questo scopo viene utilizzato il campo *rs*. Tutti gli altri campi non utilizzati sono posti uguali a 0 in quanto in un registro tutti i bit devono avere un valore.

Quindi l'esempio di istruzione *jr* dell'Assembly MIPS:

```
jr   $ra
```

viene tradotto in Linguaggio Macchina MIPS nella seguente istruzione con Formato di Tipo R:

op	*rs*	*rt*	*rd*	*shamt*	*funct*
0	31	0	0	0	8

la cui rappresentazione binaria è:

000000	11111	00000	00000	00000	001000

quindi l'istruzione in Linguaggio Macchina è data dalla sequenza:

00000011111000000000000000001000

Il Formato di Tipo R viene usato da tutte le istruzioni con una **unica Modalità di Indirizzamento:**

INDIRIZZAMENTO tramite registro

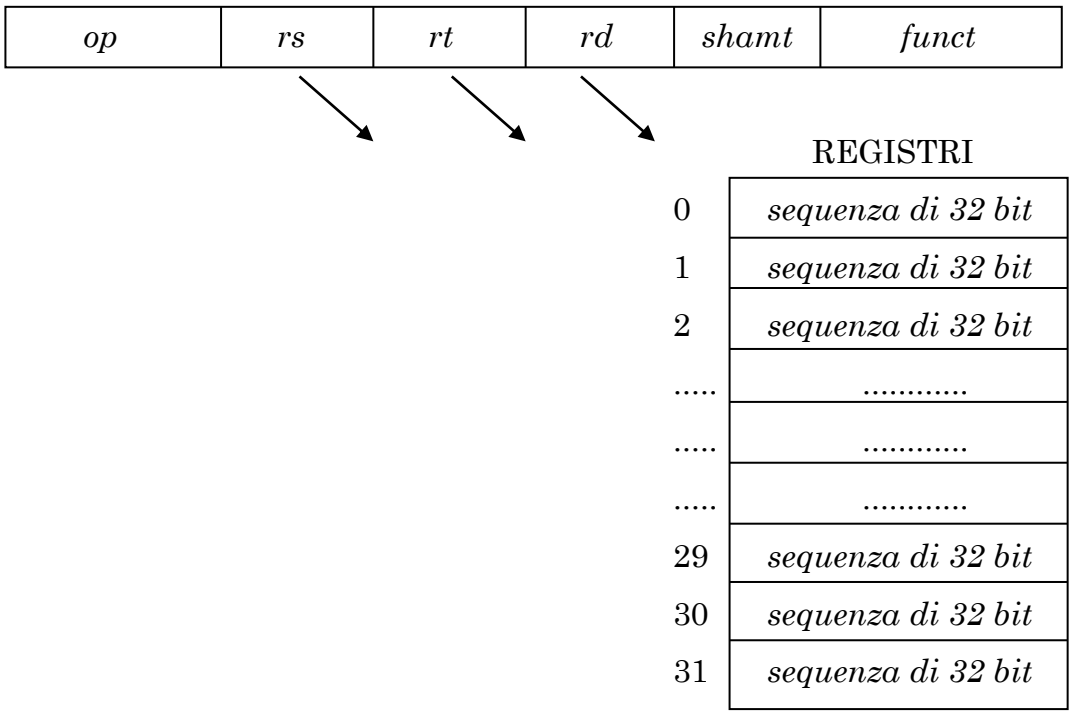
I campi di 5 bit *rs*, *rt*, *rd* del Formato di Tipo R contengono indirizzi di registri del processore in cui sono memorizzati valori di dati coinvolti nella esecuzione dell'istruzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura mostra una rappresentazione schematica del significato di tale Modalità di Indirizzamento:

INDIRIZZAMENTO tramite registro



Le seguenti Tabelle riassumono per il sottoinsieme di istruzioni presentate le regole sintattiche del Linguaggio Assembly e del Linguaggio Macchina MIPS, e la relativa Modalità di Indirizzamento.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I valori dei campi del Formato in Linguaggio Macchina sono espressi mediante la notazione decimale per i valori numerici, e la notazione simbolica per gli indirizzi dei registri del processore:

		6 bit	5 bit	5 bit	5 bit	5 bit	6 bit	esempio
istruzione	*Tipo*	*op*	*rs*	*rt*	*rd*	*shamt*	*funct*	
add	R	0	$s1	$s2	$t0	0	32	add $t0, $s1, $s2
sub	R	0	$s1	$s2	$t0	0	34	sub $t0, $s1, $s2
and	R	0	$s1	$s2	$t0	0	36	and $t0, $s1, $s2
or	R	0	$s1	$s2	$t0	0	37	or $t0, $s1, $s2
sll	R	0	0	$s2	$t0	7	0	sll $t0, $s2, 7
srl	R	0	0	$s2	$t0	7	2	srl $t0, $s2, 7
addi	I	8	$s3	$t0	–9			addi $t0, $s3,–9
lw	I	35	$s3	$t0	–9			lw $t0, –9 ($s3)
sw	I	43	$s3	$t0	–9			sw $t0, –9 ($s3)
beq	I	4	$t0	$s3	istruzioni da saltare			beq $t0, $s3, *nome*
bne	I	5	$t0	$s3	istruzioni da saltare			bne $t0, $s3, *nome*
j	J	2	indirizzo di salto incompleto					j *nome*
jal	J	3	indirizzo di salto incompleto					jal *nome*
jr	R	0	$ra	0	0	0	8	jr $ra

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Istruzione	*Formato Tipo*	*Modalità di Indirizzamento*
add	R	tramite registro
sub	R	tramite registro
and	R	tramite registro
or	R	tramite registro
sll	R	tramite registro
srl	R	tramite registro
addi	I	immediato
lw	I	tramite Base e Offset
sw	I	tramite Base e Offset
beq	I	relativo al *Program Counter*
bne	I	relativo al *Program Counter*
j	J	pseudodiretto
jal	J	pseudodiretto
jr	R	tramite registro

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).